J

SYSTEM OVERVIEW

EduMatcher

An Educational Equity Trading System

A fully functional exchange matching engine — built for learning, research, and demonstration. Real order-book mechanics, auctions, risk controls, and a ZeroMQ message bus.

J. Persson, 2026

What is EduMatcher?

A fully functional financial exchange matching engine built for education, research, and demonstration — implementing the same core mechanics that underpin real stock exchanges.

Participants connect through a gateway, type orders, and the engine matches them — publishing every fill over a ZeroMQ bus that all processes subscribe to.

Honest by design — documented teaching simplifications, not hidden shortcuts.

10	20+	5	4
Order types	pm- processes	Risk controls	Protocols

- Continuous order book
- Opening & closing auctions
- Market-maker quoting
- Price collars & circuit breakers
- Combo & OCO strategies
- OHLCV / VWAP statistics
- Drop-copy compliance feed
- Autonomous AI traders
- Real-time market data & index
- State persistence across restarts
- Multi-protocol access
- Config-driven reference data

How an order flows through the system

01

Participant

Types an ALF command at the gateway terminal to place an order.

→

02

Gateway

pm-alf-console validates and forwards it over ZMQ PUSH :5555.

→

03

Matching engine

pm-engine matches against the order book by price-time priority.

→

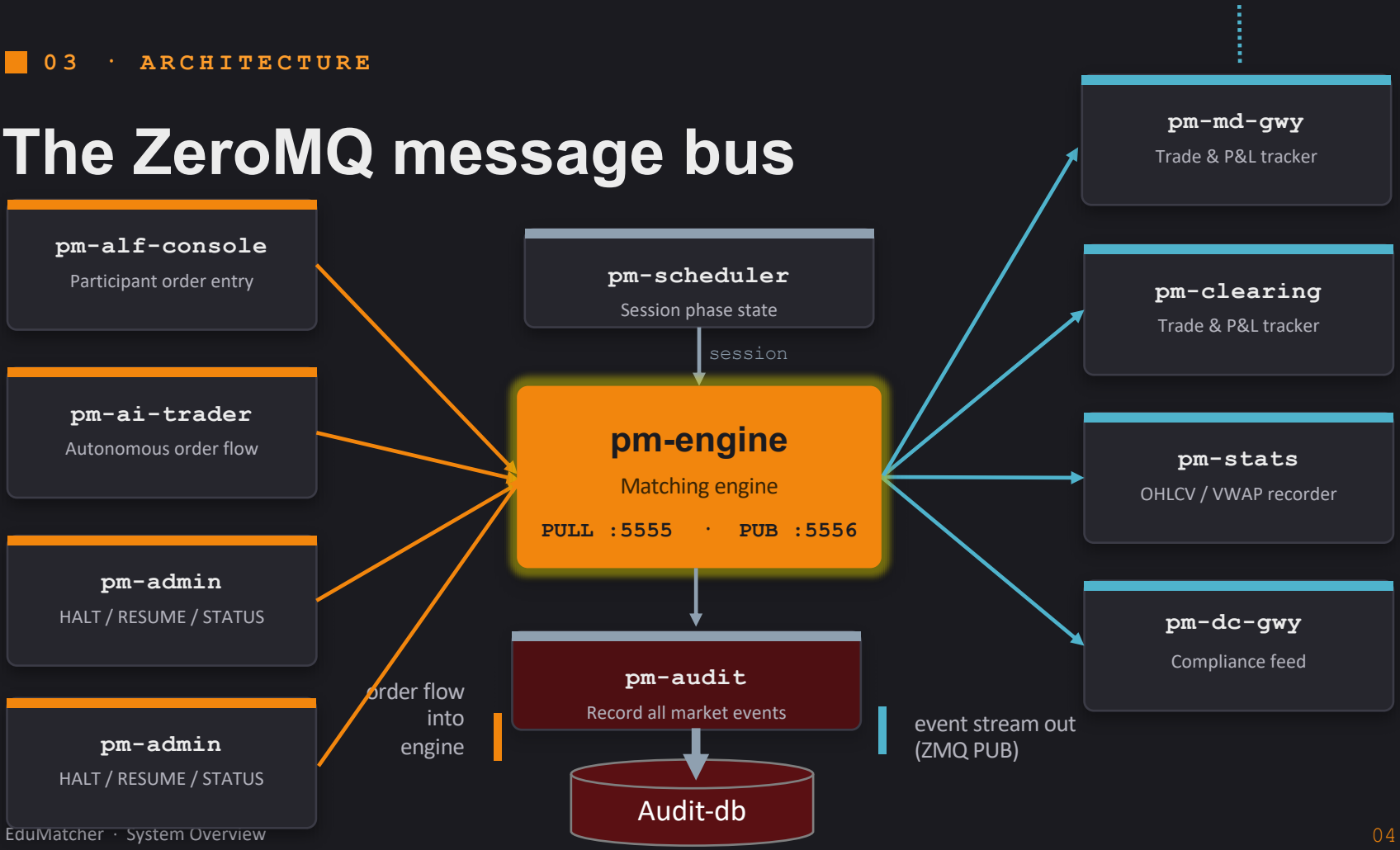
04

Event bus

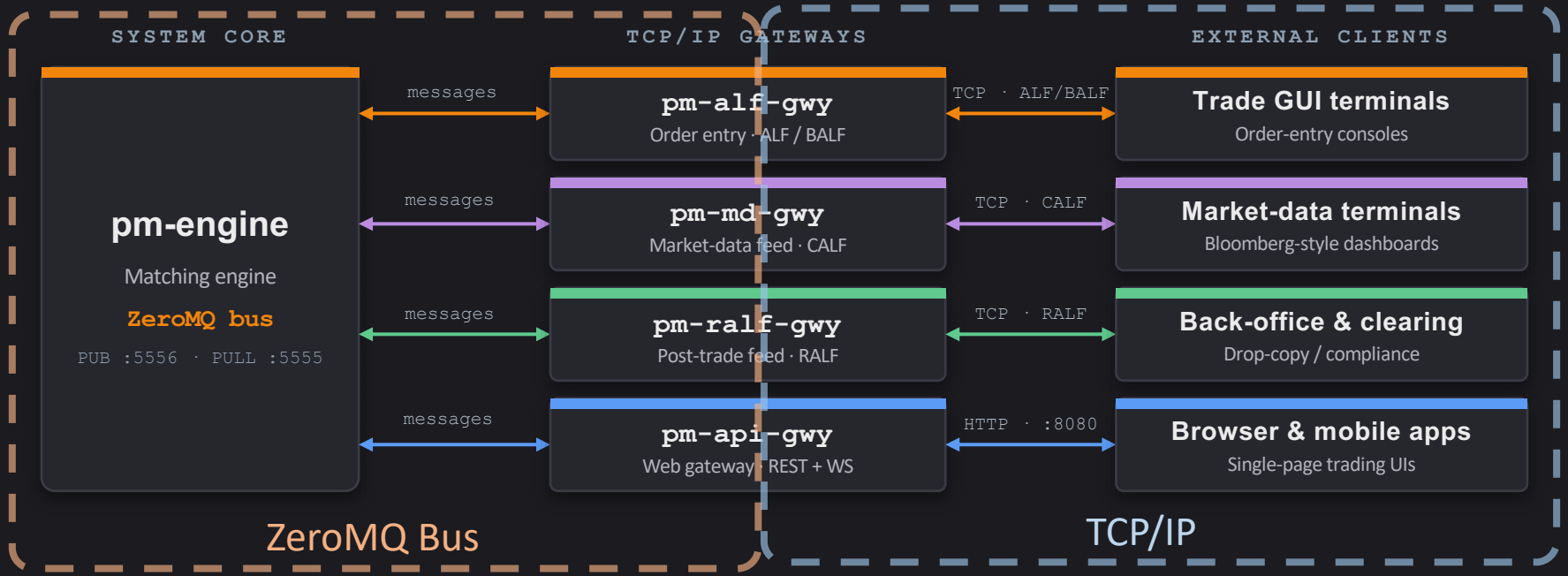
Fills are published on ZMQ PUB :5556 — every process subscribes.

ALF `NEW|SYM=AAPL|SIDE=BUY|TYPE=LIMIT|QTY=100|PRICE=150.00`

The ZeroMQ message bus



The message bus - the external view



External terminals never touch the internal bus — each gateway is a separate TCP/IP listener that speaks its protocol and bridges to the **ZeroMQ** core.

The pm- process family

Core engine & session

pm-engine — matching engine

pm-scheduler — session phases

Participant gateways

pm-alf-console — local terminal

pm-alf-gwy / balf-gwy — TCP

pm-api-gwy — REST / WebSocket

Market data & post-trade

pm-md-gwy — CALF feed

pm-ralf-gwy — post-trade

pm-index — live index

pm-calf-spy — print CALF

pm-ralf-spy - print RALF

Autonomous bots

pm-ai-trader — single bot

pm-ai-swarm — multi-agent

pm-mm-bot — market maker

Monitoring & analytics

pm-stats · pm-clearing

pm-viewer · pm-board · pm-ticker

pm-audit — event log

pm-dc-gwy — print DC

Admin & tooling

pm-admin · pm-admin-cli

pm-cverifier — config checks

config-gui — visual editor

The market maker - continuous two-sided liquidity

A market maker's job is to keep prices continuous — always showing both a bid and an ask so participants can trade at any time.

ROLE

A dedicated role & gateway

A dedicated participant that continuously quotes both sides of the book. pm-mm-bot runs the strategy through its own market-maker gateway, so its flow is tracked separately from ordinary traders.

QUOTE

Double-sided quotes

The **QUOTE** verb posts a bid and an ask together in one message and refreshes them as the market moves — maintaining a live two-sided price rather than firing one-off NEW orders.

CONFIG

Set in the symbol spec

Each symbol carries a market_maker_quotes block in engine_config.yaml. It is mandatory whenever a market-maker gateway exists, so every quoted instrument declares its MM obligation.

Teaching simplification — Market-Maker Protection (MMP) exists in the data model but isn't enforced by the engine, and it's one user per gateway.

Order types the engine supports

Aggressive

MARKET
Immediate at best price

FOK
All-or-nothing fill

IOC
Fill now, cancel rest

Passive

LIMIT
Rests on book at price

ICEBERG
Hidden reserve, visible peak

Conditional

STOP
Trigger → market

STOP_LIMIT
Trigger → limit

TRAILING_STOP
Follows price by offset

Multi-leg

COMBO
Linked legs, cascade cancel

OCO
One cancels the other

Also: AMEND live orders · Time-in-Force (DAY / GTC / GTD / IOC / FOK) · self-match prevention · combo cascade cancellation

The trading session lifecycle

Auction uncross

Opening and closing auctions accumulate orders without matching, then compute a single equilibrium price that maximizes matched volume — pm-scheduler drives every phase transition on a schedule.

Five layers of market protection

ON ADMISSION

Before an order enters the book

Instrument halt

Symbol marked HALTED by operator; resumes on command

Price collar

Order price checked vs. static & dynamic bands — else rejected

Circuit breaker

Big last-trade move auto-halts the symbol, then resumes

ON RESTING ORDERS

Acts on exposure already on the book

Kill switch

Any gateway can cancel all its own resting orders instantly

DURING MATCHING

At the instant of a cross

Self-Match Prevention

Stops a gateway trading with itself — cancel aggressor, resting, both, or allow

IN SUMMARY

One system, the whole exchange

Order book & auctions

Risk controls & market making

Analytics, clearing & compliance

AI-driven order flow on a ZeroMQ bus

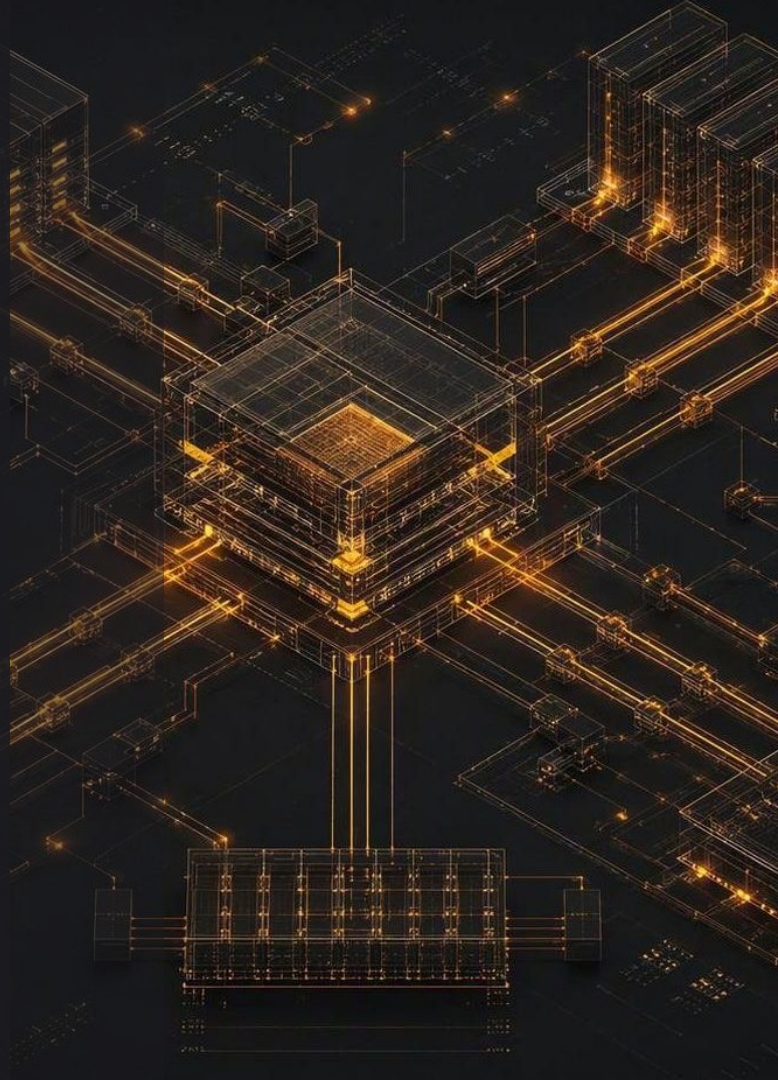
A production-grade matching engine you can read, run, and learn from.

PART TWO

Under the hood

A technical deep-dive into EduMatcher's internals

- Config & reference data
- External protocols
- Session scheduling
- Auction uncross
- Persistence & analytics
- Observability & clearing
- Known limitations



The engine reads a compiled artifact

engine_config.json

```
meta.schema_version: 3
meta.compiler_version
meta.compiled_at
meta.source_path
meta.source_sha256
meta.content_sha256
symbols:
gateways.alf:
risk_controls:
… 9 resolved sections
```

Authored once, compiled once

You author engine_config.yaml; pm-config-deploy resolves every default once and installs /ref_data/engine_config.json — the only file a running process reads.

Create → verify → deploy → load

pm-config-gen	1 · AUTHOR
pm-cverifier	2 · VERIFY
pm-config-deploy	3 · DEPLOY
pm-engine + gateways	4 · LOAD

No artifact → pm-engine starts unrestricted: no allowlists, no risk levels, no seeds.

From authored file to deployed artifact

Three stages turn intent into a running market — author the YAML, prove it is valid, then compile and install it.

1 · AUTHOR

config-gui

Browser builder with live cross-field validation

pm-config-gen

CLI generator — --symbols, --gateways, --schedule

2 · VERIFY

pm-cverifier

Read-only config checker

Verdict: OK / WARNING / ERROR

--strict fails on warnings

--format json for CI gates

Prints a Risk Summary

3 · DEPLOY

pm-config-deploy

Validates, resolves every default, then installs the artifact atomically

Restart a process to pick up a new deploy

A deploy replaces the configuration without disturbing live processes - each pm- process picks up the new artifact at its next startup.

pm-cverifier - four layers, one pass

pm-cverifier

```
--format text | json
--level info | warn | error
--strict  warnings fail
--no-color
exit 0  clean
exit 1  warnings only
exit 2  errors
CI: --strict --format json
Risk Summary always shown
Read-only — safe any time
```

Every problem in one report

The engine loader aborts on the first hard error. pm-cverifier reports them all, plus advisory warnings and a plain-English Risk Summary.

Four sequential layers

L1 YAML syntax	Y001–Y004
L2 Schema	S001–S112
L3 Semantic	M001–M025
L4 Completeness	C001–C013

Hard errors in L1 or L2 stop the later layers — fix the schema first, then re-run.

pm-config-deploy - compile and install

engine_config.json

```
pm-config-deploy cfg.yaml
--check   validate only
--show    deployed location
meta.schema_version: 3
meta.compiler_version
meta.compiled_at
meta.source_path
meta.source_sha256
meta.content_sha256
hand edits are refused
```

What compiling buys you

Defaults resolve once instead of in eight loaders that could drift — a two-key source compiles into nine complete sections.

Four guarantees on every deploy

Validated first	all 4 layers
Defaults resolved once	9 sections
Atomic install	+ source copy
Integrity checked	sha256

content_sha256 is recomputed on every load; an unknown schema_version is refused.

Who reads the artifact - and when

Every process opens the same ref-data file at its own startup and parses only its own sections.

1 · ENGINE

pm-engine

symbols · gateways.alf · risk_controls · schedule · indices · engine_tuning

pm-scheduler

schedule · country

2 · GATEWAYS

pm-alf-gwy · pm-balf-gwy
pm-ralf-gwy · pm-md-gwy
pm-dc-gwy · pm-log-srv
pm-api-gwy · pm-index

Each reads only its own section — restart it alone

3 · STARTUP

No process takes a path

The symbol universe is fixed at startup — a new symbol means a restart

Persisted prices override config seeds

A typo in a section your process never reads is invisible to it — only the owning process or pm-cverifier will catch it.

Four protocols, four jobs

ALF
Almost FIX

Order entry — human-readable text

`pm-alf-gwy`

TCP 5565

BALF
Binary ALF

Order entry — fast binary frames

`pm-balf-gwy`

TCP 5560

CALF
Channel ALF

Market-data feed — 7 channels

`pm-md-gwy`

TCP 5570

RALF
Reconciliation ALF

Post-trade — clearing / drop-copy / audit

`pm-ralf-gwy`

TCP 5580

Core ZMQ bus: `engine PULL 5555 · PUB 5556 · drop-copy 5557 · pm-index 5559`

Spy tools: `pm-calf-spy · pm-ralf-spy — live views of the CALF & RALF feeds`

ALF - the order-entry language

Almost FIX · **pm-alf-console / pm-alf-gwy** · **TCP 5565**

PURPOSE

The active order-entry protocol — how participants place, amend and cancel orders. Human-readable and the base protocol every other one is named after.

FORMAT / SYNTAX

Pipe-delimited FIELD=VALUE text, one message per line. Verbs: NEW, AMEND, CANCEL, QUOTE, KILL, STATUS, ORDERS, POS.

EXAMPLE

```
NEW|SYM=AAPL|SIDE=BUY|
TYPE=LIMIT|QTY=100|PRICE=150.00
```

TYPICAL CLIENTS

pm-alf-console (interactive REPL over stdin/stdout), the pm-alf-gwy TCP gateway, and the autonomous bots pm-ai-trader, pm-ai-swarm and pm-mm-bot.

WHERE IT'S USED

Primary order entry into pm-engine — validated, gateway-ID checked against the allowlist, then pushed over ZMQ PUSH :5555.

GOOD TO KNOW

Gateway-ID allowlisting is enforced. Supports the full order set including COMBO and OCO, all Time-in-Force codes (DAY / GTC / GTD / IOC / FOK + ATO / ATC) and self-match prevention.
DC ON/OFF control whether a Drop-Copy (DC) is sent back after each fill

BALF - binary order entry

Binary ALF · pm-balf-gwy · TCP 5560 · v1.0.0

PURPOSE

The same order entry as ALF, but as fixed-width binary frames — a low-overhead path for latency-sensitive, programmatic clients.

FORMAT / SYNTAX

Fixed-width binary frames with a per-message sequence number for ordering and gap detection. Same order semantics as ALF, not human-readable.

EXAMPLE

```
[SEQ u32][VERB u8][SYM 8]
[SIDE u8][QTY u32][PX i64]   (illustrative)
```

TYPICAL CLIENTS

Programmatic / latency-sensitive traders that need throughput over readability. Served exclusively by pm-balf-gwy.

WHERE IT'S USED

An alternative order-entry gateway into pm-engine, running alongside ALF on its own TCP port.

GOOD TO KNOW

v1.0.0 is order-entry only: no OCO, no combos, no market-data broadcast, no sequence resend/recovery and no encryption. One user per gateway.

CALF - the market-data feed

Channel ALF · pm-md-gwy · TCP 5570 · 7 channels

PURPOSE

The external market-data feed — disseminates book, trades, state, index and auction data to any downstream subscriber.

FORMAT / SYNTAX

A snapshot followed by incremental updates, multiplexed across seven named channels, with sequence-based gap detection so clients can spot a miss and re-sync.

EXAMPLE

```
SUB CALF|CH=TOP|SYM=AAPL
→ snapshot, then deltas  (illustrative)
```

TYPICAL CLIENTS

Market-data consumers, dashboards and analytics. Watch any channel live with the pm-calf-spy client.

WHERE IT'S USED

Read-only downstream distribution of everything happening in the book — subscribe to just the channels you care about.

GOOD TO KNOW

Sequence numbers per channel; on a detected gap the client re-snapshots (no resend in v1.0.0). Auction outcomes also print on the AUCTION channel.

CALF - the seven market-data channels

TOP
Best bid / offer at the top of the book

TRADE
Executed trades — the printed tape

STATE
Session phase & instrument state

INDEX
Live cap-weighted index levels

DEPTH
Full order-book depth by price level

AUCTION
Indicative & final uncross results

CB
Circuit-breaker halt / resume events

`Snapshot + incremental`
Per-channel sequence numbers; on a gap the client re-snapshots (no resend in v1.0.0).

RALF - the post-trade feed

Reconciliation ALF · pm-ralf-gwy · TCP 5580 · 3 channels

PURPOSE

The post-trade feed — a replayable trade and audit stream for clearing, drop-copy and audit consumers.

FORMAT / SYNTAX

A replayable stream across three role-based channels — CLEARING, DROP_COPY and AUDIT — with role-based entitlement governing what each consumer receives.

EXAMPLE

```
SUB RALF|CH=CLEARING
→ replayable trade stream  (illustrative)
```

TYPICAL CLIENTS

Back-office & clearing (pm-clearing), compliance / risk drop-copy consumers, and audit capture. Watch live with pm-ralf-spy.

WHERE IT'S USED

Post-trade reconciliation and compliance, downstream of the engine's fills — complementing the raw drop-copy on :5557.

GOOD TO KNOW

The stream is replayable, so a consumer can re-request and reconcile. Entitlement is role-based across the three channels.

REST API - the web gateway

REST + WebSocket · pm-api-gwy · HTTP 8080

PURPOSE

The browser-facing gateway — exposes the engine over ordinary HTTP so modern, interactive web apps can trade and stream live data without speaking the raw TCP protocols.

TYPICAL CLIENTS

Browser dashboards and single-page trading UIs, the config-gui web app, and any HTTP/JSON client — no custom protocol library required.

FORMAT / SYNTAX

JSON over HTTP. REST endpoints for request/response actions; a WebSocket channel for a persistent, real-time push of book, trade and state updates.

WHERE IT'S USED

A participant gateway into pm-engine alongside the TCP gateways, configured under the api_gateways section of engine_config.yaml.

EXAMPLE

```
POST /orders {"sym":"AAPL"...}
WS /stream → live book  (illustrative)
```

GOOD TO KNOW

REST maps order actions to HTTP verbs; the WebSocket feed bridges CALF-style market data to the browser so pages update live. Same allowlist model as the other gateways.

Building modern, interactive web apps

01

REST · actions

Place, amend, cancel and query over stateless HTTP request/response — one call, one JSON reply.

02

WebSocket · live stream

A persistent connection pushes fills, book and state changes the instant they happen — no polling.

03

In the browser

Together they drive single-page apps — live blotters, depth ladders and dashboards built with standard web tooling.

Just HTTP and JSON on port 8080 — any browser or web framework can drive the engine, no native library or raw TCP required.

Every message on the bus

Two frames per message: frame[0] = b"topic", frame[1] = JSON payload. Dotted topic names with {GW} / {SYMBOL} suffixes let each subscriber filter to exactly what it needs.

INBOUND → PULL :5555

```
system.gateway_connect
order.new
order.amend
order.cancel
order.combo
order.oco
risk.* (admin cmds)
```

OUTBOUND ← PUB :5556

```
order.ack.{GW}
order.fill.{GW}
order.cancelled.{GW}
order.expired.{GW}
trade.executed
book.{SYMBOL}
system.session / .symbols
```

SIDE CHANNELS

```
drop-copy fills
   PUB :5557
index.update
   PUB :5558
audit log — every topic
   pm-audit → audit.log
```

Topics are hierarchical, so a client subscribes to a prefix — book.AAPL, order.fill.TRADER01 — and the bus delivers only the matching frames.

The socket & port map at a glance

Internal ZeroMQ sockets bind to localhost; the five TCP gateways are the only externally reachable listeners — the boundary between the core and the outside world.

CORE ZMQ SOCKETS (localhost)

PULL	:5555	orders in
PUB	:5556	events out
PUB	:5557	drop copy
PUB	:5558	index publish
PULL	:5559	index admin

TCP GATEWAYS (external)

pm-alf-gwy	:5565	ALF text order entry
pm-balf-gwy	:5560	BALF binary order entry
pm-md-gwy	:5570	CALF market data
pm-ralf-gwy	:5580	RALF post-trade
pm-api-gwy	:8080	REST + WebSocket

pm-engine is the single writer on :5555/:5556/:5557; pm-index adds :5558/:5559. Each gateway translates one external protocol onto that internal bus.

Reading the wire with the spy tools

Passive, read-only taps connect exactly like a real client and print every decoded frame — so you can learn or debug a protocol without standing up a full terminal.

pm-calf-spy

Taps the CALF market-data feed

pm-md-gwy · **TCP :5570**

Decoded frame types

WELCOME · SNAP · MD · TRADE

STATE · IDX · DEPTH · AUCTION

CB · HB · ERR

pm-ralf-spy

Taps the RALF post-trade feed

pm-ralf-gwy · **TCP :5580**

Roles: CLEARING / DROP_COPY / AUDIT

Decoded frame types

WELCOME · SNAP · EXEC · EOD

HB · EXIT · ERR

Because the spies speak the real wire protocol, whatever they print is exactly what a Trade-GUI or market-data terminal would receive — the ground truth for protocol work.

The compliance & audit plane

Three independent consumers ride the same PUB :5556 event stream — RALF exposes them as CLEARING / DROP_COPY / AUDIT roles — giving the exchange a replayable system of record.

DROP COPY

Engine PUB :5557

Per-participant fill stream

Sequence numbers + gap replay

so a client can detect and

re-request any missed fill

Bound lazily at engine start

AUDIT TRAIL

pm-audit · SUB :5556

Passive logger of every

topic and payload

`data/audit.log`

`[TS] [TOPIC] {JSON}`

Authoritative, replayable record

CLEARING

pm-clearing · SUB :5556

Settlement + VWAP P&L

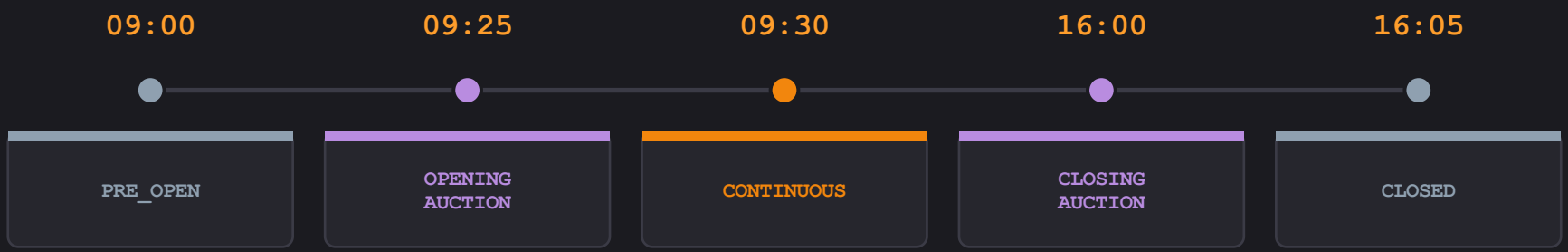
per gateway

`data/clearing.db`

Realized & unrealized positions persisted

None of these can alter the book — they only subscribe. The audit log is the single source of truth for reconstructing exactly what happened in a session.

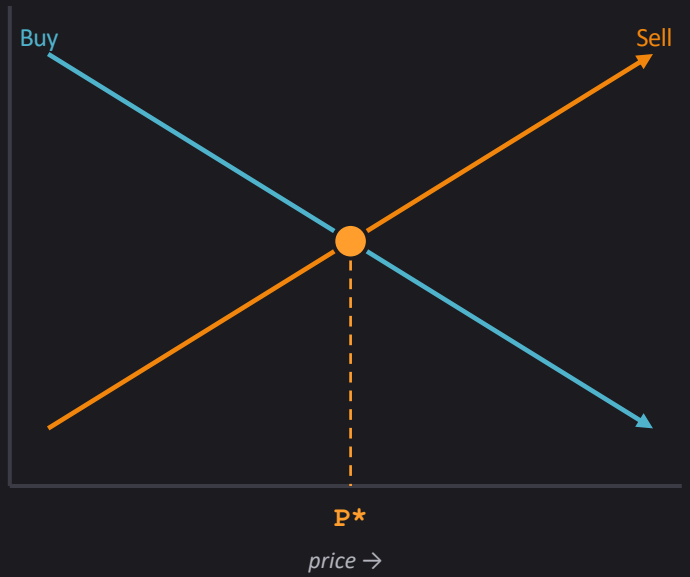
Scheduling the trading day



pm-scheduler

pushes session.transition over a ZMQ PUSH socket at each wall-clock time. Config keys: pre_open · opening_auction_start · continuous_start · closing_auction_start · closing_auction_end. Flags: --now (fire all), --daily, --delay. sessions_enabled=false pins the market to CONTINUOUS.

How an auction uncrosses



1	**Accumulate**	Auction phases rest orders — no matching yet.
2	**Build curves**	Cumulative buy (desc) & sell (asc) quantity per price.
3	**Maximize**	P* = argmax executable = min(buy, sell) at each price.
4	**Tie-break**	Smallest surplus, then lowest price.
5	**Uncross**	All fills at the single P* — no slippage.

Result published on auction.result.{symbol} — price, matched qty, surplus & imbalance side.

What survives a restart

Engine state — rewritten on clean shutdown, reloaded at startup

`gtc_orders.json`

Resting GTC orders — original price-time priority preserved

`gtc_combos.json`

GTC combo parents & child-leg links

`book_stats.json`

last_buy / last_sell / prev_close per symbol

Accumulating stores — grow across sessions

`stats.db`

OHLCV · snapshots

`clearing.db`

positions · P&L

`audit.log`

full event log · 10 MB × 5

`indexes/*.jsonl`

index history

Ctrl-C writes state and emits system.eod; SIGKILL skips it. Malformed JSON → empty state, startup never fails. Order IDs are UUID4, stable across restarts.

Statistics, reporting & the index

pm-stats → stats.db

Subscribes to trades, book & index updates. Records OHLCV daily stats and 15-min intraday snapshots.

Tables

`daily_stats`
`price_snapshots`
`trade_log`
`order_events`
`index_daily_stats`
`index_level_snapshots`

pm-stats-cli: daily · trades · order-lifecycle · index-daily (table / JSON / CSV)

pm-index · live market index

```
index_level = aggregate_cap / divisor
```

- Cap-weighted across constituents
- Corporate actions: split · dividend · shares · add · delist
- Divisor initialised to base_value
- Publish throttle ~1s · intraday OHLC reset at open

How the market index is built

pm-index is a standalone process that turns the trade stream into a live, divisor-based index and republishes it for market-data clients.

METHODOLOGY

Market-cap weighted, using

a divisor · base_value 1000.0

Each constituent carries

outstanding_shares

Falls back to config

reference prices if it misses

early trades

DATA FLOW

In ← engine PUB :5556

trade.executed,

session.state, system.eod

Out → PUB :5558

index.update → CALF INDEX

channel via pm-md-gwy

Admin ← PULL :5559

CORPORATE ACTIONS

pm-index-admin-cli

Splits, dividends, issuance,

buy-backs; add / delist

--dry-run + confirm prompt

pm-index-cli

Read-only history queries

pm-index never writes to the engine — it only listens and republishes. --reset clears the divisor and last prices to rebaseline cleanly from config reference prices.

Watching the market in real time

Everything is a published event

```
order.ack.*
order.fill.{gateway}
trade.executed
book.* · depth.*
quote.* · combo.status.*
session.state · session.transition
auction.result.{symbol}
circuit_breaker.halt/resume.*
index.update · system.eod
```

Spy & monitor clients

pm-viewer
single-symbol book

pm-board
multi-symbol board

pm-ticker
tape + OHLCV

pm-orders
cross-gateway status

pm-audit
full event capture

pm-calf-spy
market-data feed

Drop-copy on :5557 streams every fill for risk & back-office (no auth).

Multi-leg combo orders

FAILED

Leg cancelled/expired → cascade-cancel siblings

CANCELLED

Explicit CANCEL|COMBO_ID=

REJECTED

Validation failure

First-class, not synthetic

2–10 legs · each a different symbol · AON only (ANY / MIN reserved) · legs rest as normal LIMIT orders on their own books · executed fills are never unwound · GTC combos persist to gtc_combos.json.

Positions, VWAP & P&L

Average cost (VWAP)

```
(avg·|net| + price·qty) / |net'|
```

On every position-increasing fill

Realized — long close

```
+= (price − avg_cost) · qty
```

SELL that reduces a long

Realized — short close

```
+= (avg_cost − price) · qty
```

BUY that reduces a short

Unrealized (mark-to-market)

```
net_qty · (mark − avg_cost)
```

Live exposure at mark price

pm-clearing writes clearing.db per trade — trade_events · gateway_symbol_positions · gateway_daily_summary. Query: pm-clearing-cli positions · pnl · exposure · reconcile.

Operating the exchange

Every operator action is a two-frame ZeroMQ command — b"topic" + JSON over PUSH :5555, with acks on SUB :5556 — wrapped by one shared execute command core.

THREE ENTRY POINTS

pm-admin

Interactive ADMIN console

pm-admin-cli

One-shot command, exit 0/1

ExchangeCommandClient

Python library for scripts

THE COMMAND SET

HALT · RESUME
HALT_SYM · RESUME_SYM
CANCEL_SYM · KILL|GW=
QCANCEL · ORDERS · BOOK
SESSION|STATE= · SCHEDULE
SESSION_STATUS · HELP

ADMIN role required to mutate state

The PUSH command socket has no authentication — --id is a label, not a credential — so real access control lives at the network boundary, not in the protocol.

Filling the book without humans

Autonomous processes connect as ordinary gateways over PUSH :5555 / SUB :5556 — the order book cannot tell a bot from a person.

pm-mm-bot

Autonomous MARKET_MAKER

Posts a two-sided quote and auto-reprices on:

- a fill on either side
- mid-price drift
- session-state change

One symbol per instance

pm-ai-trader

Single autonomous trader

Loop: risk-pause → freshness → pick → price → side → submit

Resting limit orders only

never market, FOK or IOC

pm-ai-swarm

Swarm launcher — spawns N bots in one command, distributing profiles + symbols

Profiles

aggressive · balanced · passive

Run one pm-mm-bot per symbol (or several competing with --id-suffix) and a swarm of AI traders to generate a realistic, continuously moving book for testing and demos.

Limitations that matter most

The constraints most likely to change what you can actually do — know these before you rely on it.

HIGH IMPACT

BALF 1.0.0 gaps

No OCO, no combos, no market-data broadcast, no sequence resend, no encryption.

HIGH IMPACT

No trade unwind

Fills are irrevocable — a failed combo cancels unfilled legs but never reverses executed trades.

HIGH IMPACT

No implied orders

COMBO is native, not synthetic — the engine derives no implied liquidity.

Minor limits & teaching scope

Deliberate scope choices for a readable teaching engine — documented, not accidental, and easy to add later.

LOW IMPACT

No auth / authz

Raw ZMQ bus & side feeds (5557, 5559) are open — fine on a local teaching box, not for shared use.

LOW IMPACT

No combo-order book

Combo legs post to each symbol's own book; there is no dedicated spread book across them.

LOW IMPACT

Teaching simplifications

MMP is data-model only (not enforced); one user per gateway.

Exceptionally well-supported

800 pages

User guide — the complete reference manual

45 chapters

Training course with hands-on guided exercises

✓ Quizzes to check your knowledge

✓ Self-study question sets

✓ Full original design documentation

✓ All source code, free to use

`github.com/johan162/EduMatcher`

Try it yourself in a Linux VM

We ship a prepared Multipass image of EduMatcher — spin up the whole pre-configured system in a fresh Linux VM and start exploring in minutes, with no manual install.

GET STARTED — TWO COMMANDS

```
$ multipass launch edumatcher --name edu
$ multipass shell edu
```

Pre-configured

The whole system already set up — services, config and sample data baked in via cloud-init.

Any OS

Runs on Windows, macOS and Linux using each platform's native hypervisor (Hyper-V, QEMU).

Requires Multipass

Install Canonical Multipass first (e.g. snap install multipass), then launch — nothing else to set up.

THE TAKEAWAY

Read it. Run it. Extend it.

Config-driven — one YAML defines the whole market

Protocol-rich — ALF, BALF, CALF, RALF + REST/WS

Fully observable — every change is a bus event

Honest about its limits — documented scope

A real matching engine, small enough to read end to end.